3.7. OWASP Top 10

3. Сети, базы данных,
безопасность кода

В этом уроке

- Основы OWASP Top 10: ключевые уязвимости веб-приложений (SQL-инъекции, XSS, SSRF, небезопасная аутентификация).
- Принципы эксплуатации уязвимостей: как злоумышленники используют слабые места.
- Методы защиты: параметризованные запросы, экранирование ввода, валидация данных, обновление зависимостей.
- Практика: реализация HTML-формы с защитой от SQL-инъекций и XSS, ограничение по времени.

Демонстрация SQL-инъекции

Форма логина:

```
SELECT * FROM users WHERE login = '$login' AND password = '$password'
```

👉 Вводим в поле логина:

```
' OR '1'='1
```

📌 Результат: вход без пароля

✅ Исправление:

```
cursor.execute("SELECT * FROM users WHERE login=? AND password=?", (login, password))
```

XSS — межсайтовый скриптинг

Что такое XSS:

- Вставка вредного скрипта в страницу

Пример:

```
<script>alert('XSS!')</script>
```

⚠️ Что может случиться:

- Кража cookies, подмена страницы, фишинг

✅ Как защититься:

- Экранировать HTML
- Запрещать выполнение JS из пользовательского ввода

XSS — межсайтовый скриптинг

Форма комментария:

```
<input name="comment">
```

Вводим:

```
<script>alert("Я внутри!")</script>
```

Результат:

🛑 Скрипт выполняется на странице

✅ Исправление:

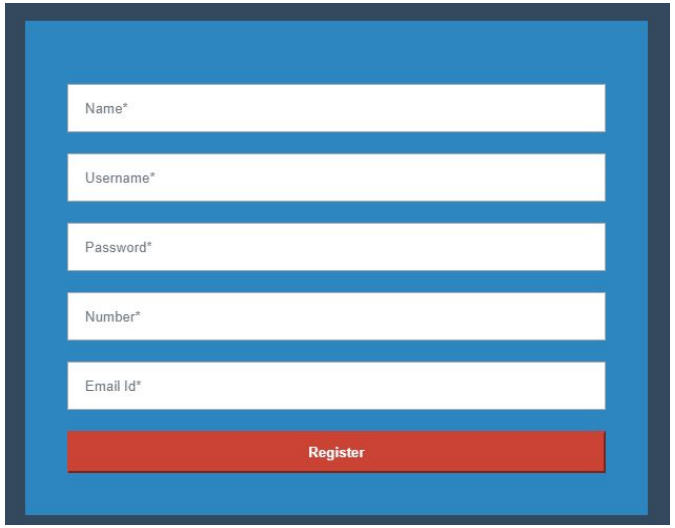
- Использовать шаблонизатор, который автоматически экранирует HTML
- Либо вручную:

```
import html
safe_text = html.escape(comment)
```

Защита форм — лучшие практики

- Всегда валидируйте и проверяйте данные:
  - Типы, длину, список допустимых значений
- Используйте POST, а не GET, для критичных операций
- Вводите ограничения по времени и количеству попыток
- Используйте CAPTCHA при необходимости
-

```
if not username.isalnum():
    raise ValueError("Неверный логин")
```



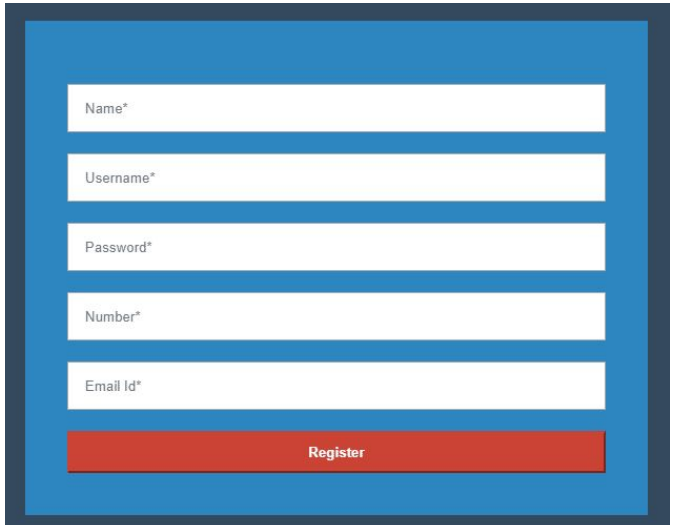
Практическое задание

Напишите HTML-форму логина

Напишите Python-скрипт, который проверяет вход

Сделайте:

- фильтрацию SQL
- защиту от XSS
- ограничение по времени



Скринкаст

Итоги урока

- Научились распознавать и предотвращать атаки из OWASP Top 10.
- Освоили защитные техники: подготовленные SQL-запросы, экранирование пользовательского ввода, валидация данных.
- Применили навыки на практике: создали защищённый Python-скрипт для обработки логина и формы.
- Поняли, как ограничивать риски через ограничения по времени и контроль доступа.